ASSIGNMENT - 1

Name: Riyaz Kamal

Register No: 192572049

Course: DSA0117A

GitHub repository

https://github.com/Riyazkamal78/DSA0117A_assignments

Scope and assumptions

The original system charges by the farthest zone travelled. Inner, Middle and Outer cost 1000, 2000 and 3000 paise. The morning peak is 08:00 inclusive to 10:00 exclusive; the evening peak is 17:00 inclusive to 20:00 exclusive. A valid accepted journey leaves at least 5000 paise. Invalid or denied requests leave the balance unchanged.

The brief does not set concession rates. This implementation assumes 10% for students and 20% for senior citizens, applied after the peak or weekend adjustment. Percentage results are rounded half up to the nearest paisa at each stage. These rates and rounding rules need approval before operational use.

For the extension, the weekend discount replaces the peak surcharge. A monthly pass covers its two selected zones for one YYYYMM calendar month. Every zone traversed must be covered, including intervening zones. A zero-fare pass journey must still satisfy the Rs 50 reserve. The pass purchase price, refunds, top-up payment integration and real calendar validation are outside scope.

Report map

Task	Subject	Evidence
1–3	Objects constants and types	UML, runnable demonstrations, measured sizes
4–5	Conversions and operators	Recorded predictions, actual output, explanations
6–8	Control structures classes and input	Completed given code, two styles, input recovery
9–10	Validation and measured totals	24 checks, scale and precision measurements
11–12	Recommendation and adaptation	Configuration experiment and 11 revised checks
Appendices	Shared implementation and all sources	Complete code and supplementary execution images

All shared constants and card behaviour are defined in Common/metro.hpp. Task06 preserves the supplied standalone program outside the completed TODO blocks. Task04 and Task05 retain the supplied program logic. The remaining tasks demonstrate or test the shared design.

The PDF asks for a repository named DSA01-192572049-Assignment1 and progressive commits. The repository shown on the cover is the one specified for this submission package. Repository naming and authentic progressive work remain submission conditions to confirm with the faculty.

Task 1 Understand the problem and identify the objects

Retain SmartCard, Journey and FareTable as separate responsibilities. The card owns money, a journey records a request, and the fare table computes a price. This prevents a fare revision from requiring edits to balance-handling code.

Object	Data	Behaviour and justification
SmartCard	Private balance in paise	Checks reserve, deducts and recharges; one owner protects stored value.
Passenger	Category	Selects a concession through the fare policy; no duplicate money state.
Journey	Farthest zone, hour, passenger	Captures a complete fare request; validation precedes charging.
Zone	Scoped enumeration	Names legal fare bands and prevents accidental mixing with ordinary integers.
FareTable	Central base fares and percentages	Quotes a fare; separates changeable tariff rules from card storage.

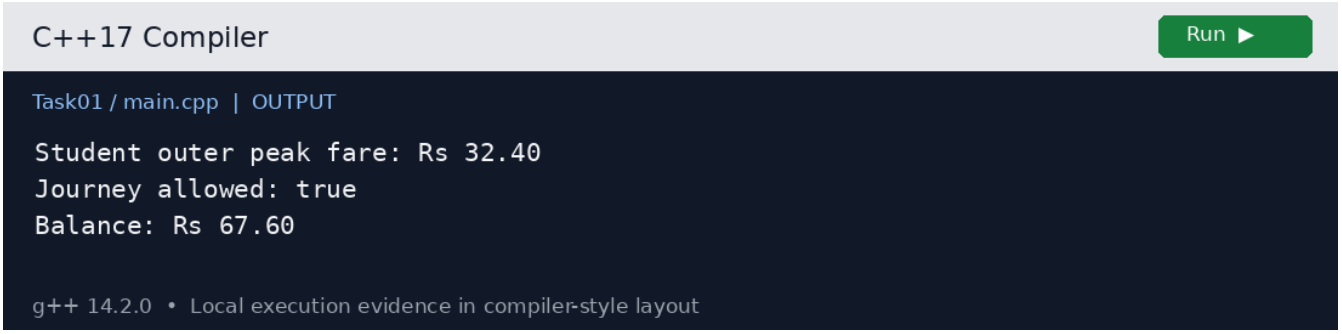
OOP principles and correctness rules

Encapsulation makes balance_ private and restricts writes to validated operations. Abstraction lets callers use quote(), canTravel() and deduct() without manipulating storage. Inheritance could introduce StudentPassenger and SeniorPassenger subclasses if their behaviours diverged. A virtual FarePolicy

interface could support interchangeable tariff algorithms through runtime polymorphism. The current small category set needs neither hierarchy; an enum and composition are simpler.

Every request must have a known zone, valid category and hour from 0 to 23. Fare must be nonnegative and exactly represented in paise. A successful deduction occurs once and leaves at least Rs 50; rejection changes nothing. Recharges must be positive and must not overflow. Peak and concession rules must be applied in a documented order. These rules prevent the reported negative balance and inconsistent boundaries.

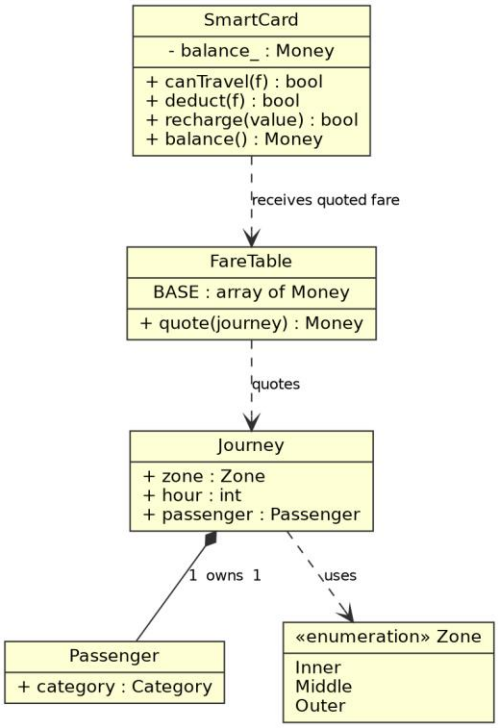
Execution output

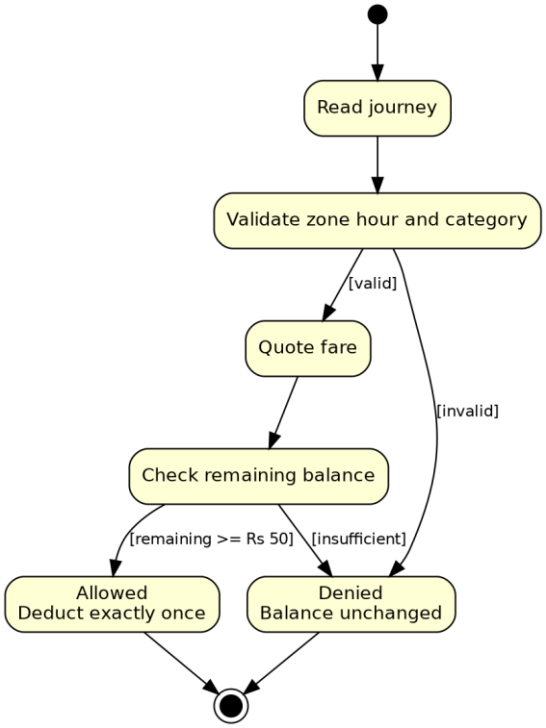


Task 1

Alternative concept and complexity

A procedural design can store a balance variable and use quote(zone, hour, category), canTravel(balance, fare) and deduct(balance, fare) free functions. Each journey still takes O(1) time and each card O(1) memory. A centralized procedural policy can also change a tariff in one place; OOP does not automatically reduce edits. The class advantage is that unrelated code cannot write an invalid balance. A scattered global design requires finding every mutation and duplicated fare expression; change cost grows with the number of duplicated sites.





UML class diagram of the original design

UML state view of one journey request

Task 2 Use tokens keywords identifiers and constants correctly

Use constexpr typed constants for the base fare array, reserve and percentages. A tariff revision changes BASE or the appropriate named percentage once in Common/metro.hpp. Zone is an enum class, so using a zone as an array index requires validation and an explicit conversion.

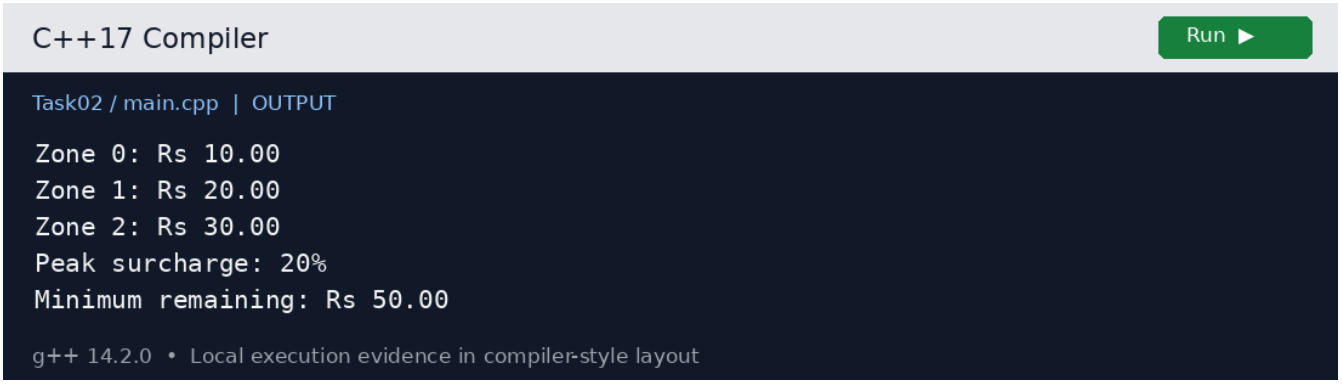
Token kind	Examples	Purpose
Keyword	class, return, if, constexpr	Reserved C++ grammar; cannot name a user variable.
Identifier	SmartCard, balance_, BASE	Names declared by the program.
Constant or literal	5000, 20, true	Fixed values; named constexpr objects express their meaning.
Operator	+, <=, &&, -=	Computes amounts, combines guards and updates state.

Token kind	Examples	Purpose
Punctuator	; { } ( ) ,	Terminates statements, groups blocks and separates arguments.

An identifier begins with a letter or underscore, followed by letters, digits or underscores; it cannot be a keyword and is case-sensitive. Prefer descriptive names, PascalCase types, consistent function names and a trailing underscore for private data. Avoid leading underscores, especially double underscores or underscore followed by a capital, because reserved-name rules can apply.

#define is preprocessor text substitution without a C++ type or namespace scope. const prevents modification after initialization but need not be a compile-time constant. constexpr requires constant-expression initialization for these objects and allows static_assert checks. A plain enum exposes its enumerator names in the enclosing scope and implicitly converts to an integral type; enum class scopes the names and avoids that implicit conversion.

Execution output



Task 2

Alternative concept and complexity

A macro version could use #define INNER_PAISE 1000 and integer zone codes. Arithmetic remains O(1) and preprocessing itself uses no runtime storage. A constexpr scalar often also needs no storage unless its identity is used. The constexpr array can occupy storage when indexed at runtime; here three Money entries require 24 bytes on the measured platform. Macro substitution is not guaranteed to be faster and loses type and scope checks. Do not duplicate constants in application code.

Task 3 Choose data types and check type compatibility

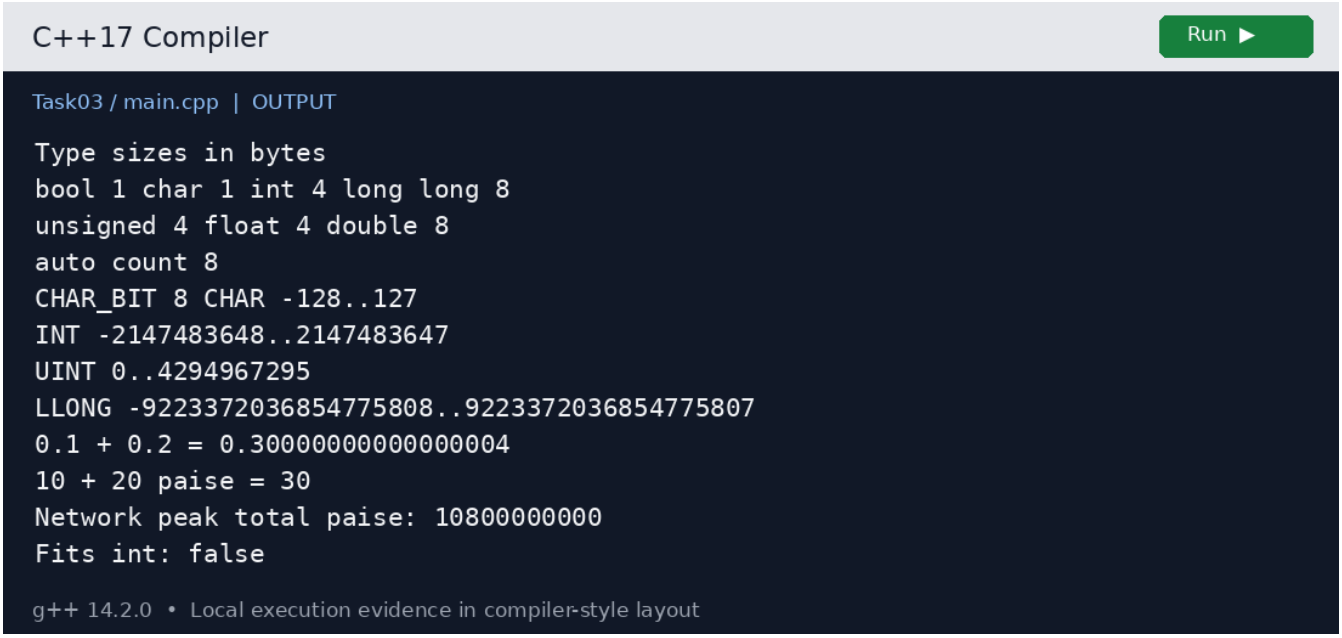
Value or type	Choice and measured size	Reason or range
Fare and balance	long long, 8 bytes	Signed paise; LLONG_MAX = 9223372036854775807.
Hour and percentage	int, 4 bytes	Hour 0–23; int range −2147483648 to 2147483647.
Journey count	long long, 8 bytes	3 million fits int, but wide products prevent overflow.
Flags	bool, 1 byte	Peak and allowed values are false or true.
Character	char, 1 byte	Measured signed range −128 to 127; CHAR_BIT = 8.
Unsigned count/index	unsigned, 4 bytes	0 to 4294967295; validate signed input before conversion.
Approximate values	float 4; double 8 bytes	Binary floating point; not selected for stored money.
Zone	enum class Zone	Named legal bands; explicit checked conversion to index.
Inferred count	auto count = 3000000LL	Deduces long long, 8 bytes; auto is not a new numeric type.

Money in integer paise avoids representing decimal fractions in binary. The measured expression 0.1 + 0.2 prints 0.30000000000000004, whereas 10 + 20 paise gives exactly 30. Casting an already rounded double total into an integer does not recover lost precision. Calculate in paise from the start and round percentage operations explicitly.

In int + double + bool, bool promotes to int and integer operands are converted to double for arithmetic with double. Assigning the result back to int truncates its fractional part if representable; a narrowing brace initializer rejects such implicit loss. Type compatibility is not a guarantee of business correctness.

A network peak collection of 3,000,000 × 3600 = 10,800,000,000 paise exceeds INT_MAX. Even 3,000,000 × 1000 = 3,000,000,000 paise exceeds it. A long long multiplication must be selected before arithmetic, for example n * 3600LL. Signed int overflow is undefined behaviour, so the experiment checks bounds instead of relying on a wrapped result.

Execution output



Task 3

Alternative concept and complexity

A double-rupee solution would add fareRupees to a double total and format with fixed and setprecision(2). Addition is O(1), just as integer addition is, and both types occupy 8 bytes here. Formatting hides small errors but does not remove them. Task10 measures repeated Rs 10.01 additions: error changes with n and is not necessarily monotonic. Actual whole-rupee fares in the adult demonstration add exactly at these scales. Both representations still need range checks.

Task 4 Control implicit conversions

Prediction recorded before execution

3 3.5

C 68

1 true

s >= u

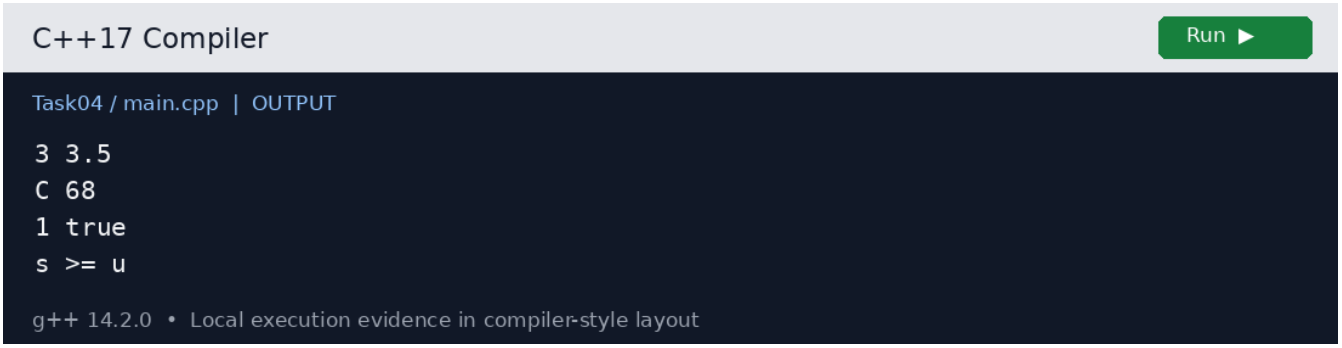
The prediction matched execution on the measured ASCII-compatible system. The prediction file was recorded before this preparation run.

Line	Explanation
3 3.5	a / b uses integer division first, producing 3, then converts it to double. Casting a to double makes b convert to double and produces 3.5.
C 68	'A' promotes to int in + 2; the representable result converts to char C. In ch + 1, ch promotes back to int, producing 68 on ASCII.
1 true	Nonzero int 5 converts to bool true. Default insertion prints 1; boolalpha changes subsequent bool formatting to true or false.
s >= u	int and unsigned int have equal rank here. −1 converts to unsigned int as UINT_MAX, which is greater than 1. The condition is false.

The compiler emits -Wsign-compare at s < u. The explicit version preserves the surprising result with static_cast<unsigned int>(s), then demonstrates the intended mathematical comparison by converting both operands to long long. Blindly adding casts only hides warnings; choose a representation that preserves the intended values.

Other conversion points include int hour comparisons, enum-to-int conversion, int-to-size_t indexing after validation, Money-to-double conversion used only for display, and bool-to-counter conversion in tests. Percentage arithmetic keeps Money operands integral. The expression int x{3.7}; is compiled separately and rejected as narrowing, with its diagnostic retained in the task folder.

Execution output



Task 4

Alternative concept and complexity

The explicit.cpp listing uses braces for initialization and static_cast at conversion points. It has O(1) time and O(1) space like the original; casts do not imply dynamic allocation. Braces catch implicit narrowing at compile time, while explicit casts can still request a lossy conversion and require review. Signed-to-unsigned conversion remains defined even if undesirable. Replacing every conversion mechanically is less useful than making the intended numeric domain clear.

Task 5 Write correct expressions with the right precedence

Prediction recorded before execution

11 16

9 6 4

6 4

3 1 -3 -1

18

Output	Operators and evaluation
11 16	Multiplication binds before addition: 5 + 3 × 2 = 11. Parentheses force 8 × 2 = 16.
9 6 4	x++ yields 5 then increments x to 6; ++y changes y to 4 and yields 4. Their sum is 9. Different variables avoid an unsequenced same-object update.

Output	Operators and evaluation
6 4	The relational test chooses x = 6 through ?: . Remainder and multiplication share precedence and group left to right: (6 % 4) × 2 = 4.
3 1 -3 -1	Integral division truncates toward zero. Remainder has the sign of the dividend, so −10 = (−3) × 3 + (−1).
18	The loop visits i = 0, 3, 6, 9. Compound assignment accumulates 0 + 3 + 6 + 9. The next i is 12, so the condition fails.

The peak expression is (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20). Comparisons bind before &&, and && binds before ||. Parentheses make the two windows visible even though the unparenthesized form is equivalent. && and || short-circuit; precedence describes grouping, not a general left-to-right operand evaluation rule.

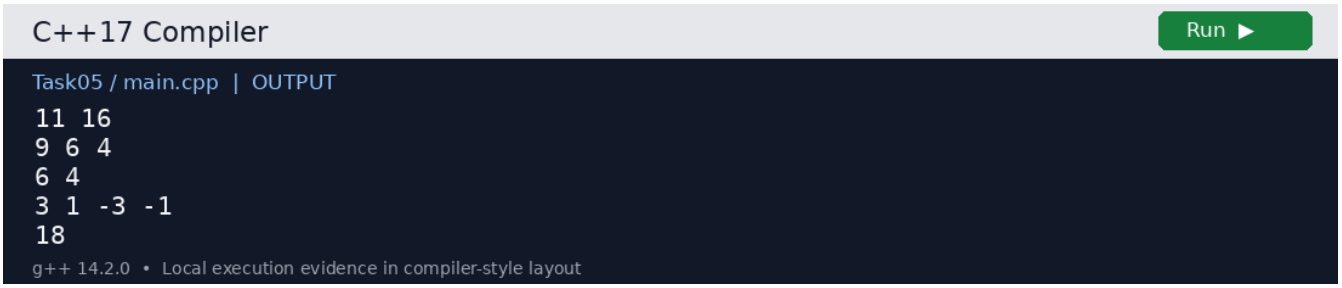
gross = roundPercent(base, peak ? 100 + PEAK_PERCENT : 100);

fare = roundPercent(gross, 100 - concession(category));

balance -= fare;

The conditional operator selects one multiplier; addition in its middle expression forms 100 + PEAK_PERCENT. The conditional operator groups right to left when nested, so parentheses help with multiple tariff choices. Compound assignment updates balance once and binds less tightly than arithmetic. Prefix ++x yields the new value; postfix x++ yields the old value. Avoid i = i++: in C++17 it assigns the old value back, defeating the increment; before C++17 that expression had undefined behaviour.

Execution output



Task 5

Alternative concept and complexity

A validated constexpr array<bool, 24> lookup is O(1), as is the expression. peak_table.cpp tests all 24 hours with zero mismatches; the table occupies 24 bytes. Unchecked invalid indices are unsafe. Prefer the expression for two intervals and a table for irregular periods.

Task 6 Design the control structures

Use a switch for a small closed zone set, if–else for invalid input and reserve decisions, and ?: for selecting a compact surcharge multiplier. Include a default branch: without a matching case or default, a switch simply executes no case body, so a required fare might remain unset. Task06 returns an invalid sentinel and canTravel rejects it.

Algorithmic pseudocode

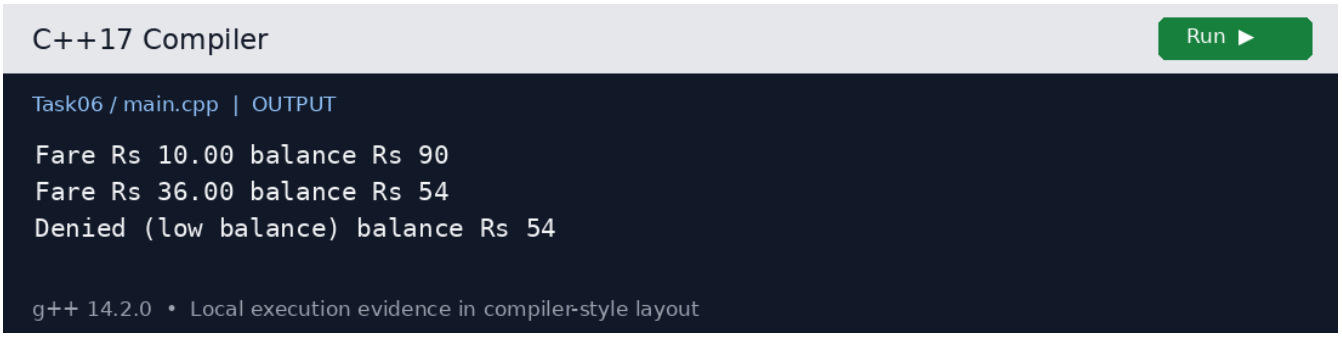
```
ALGORITHM PROCESS_JOURNEYS
INPUT journeys, initialBalance
OUTPUT fare and remaining balance or rejection for each journey
BEGIN
  balance ← initialBalance
  FOR EACH journey IN journeys DO
    IF zone OR hour OR category is invalid THEN
      OUTPUT invalid journey
      CONTINUE
    END IF
    SELECT zone
      CASE Inner:  base ← 1000
      CASE Middle: base ← 2000
      CASE Outer:  base ← 3000
    END SELECT
    peak ← (8 ≤ hour < 10) OR (17 ≤ hour < 20)
    gross ← ROUND_PERCENT(base, IF peak THEN 120 ELSE 100)
    fare ← ROUND_PERCENT(gross, 100 − concessionPercent)
    IF balance ≥ 5000 AND fare ≤ balance − 5000 THEN
      balance ← balance − fare
```

```
        OUTPUT fare, balance
    ELSE
        OUTPUT denied, balance
    END IF
  END FOR
END
```

A range-based for loop is best for the fixed array because it avoids index bookkeeping and visits every Trip once. An indexed for loop is useful when reporting a sequence number or processing parallel arrays. A while loop suits an incoming stream whose length is unknown. continue skips invalid requests; break exits a switch case or ends an input loop on an explicit stop condition. Do not break the entire day's processing just because one journey is denied.

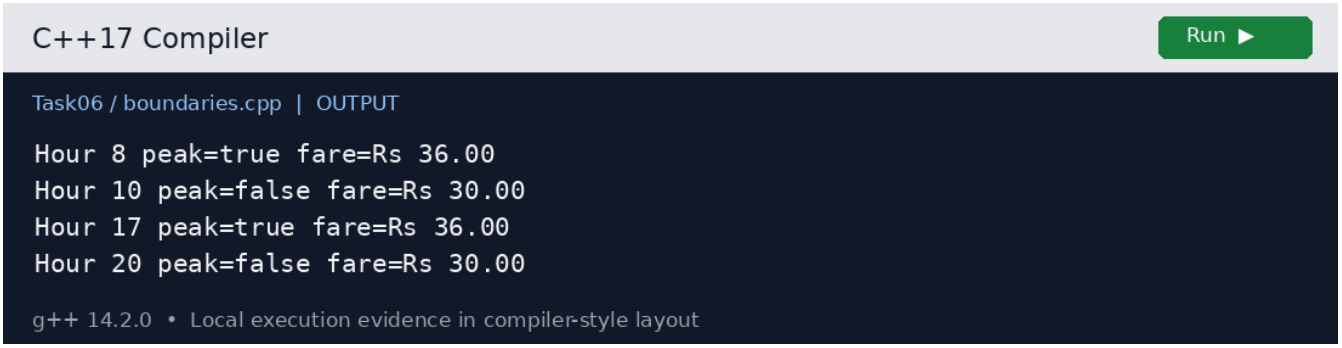
The supplied program is completed only inside TODO 1 and TODO 2. It computes the three fares with integer paise and matches all three expected lines exactly. The separate boundary program quotes Outer at 08:00, 10:00, 17:00 and 20:00 as Rs 36, Rs 30, Rs 36 and Rs 30 respectively. The shared policy also applies the assumed concessions.

Execution output



Task 6

Execution output



Alternative concept and complexity

An array lookup replaces the switch with FARE.at(validatedIndex). Both are O(1); a compiler may turn a switch into a jump table. A long long array with three values uses 24 bytes here; the supplied int array uses 12. A fourth zone needs a new enum and fare entry in either design, but the switch also needs another case. A variable-size configuration table costs O(z) space for z zones.

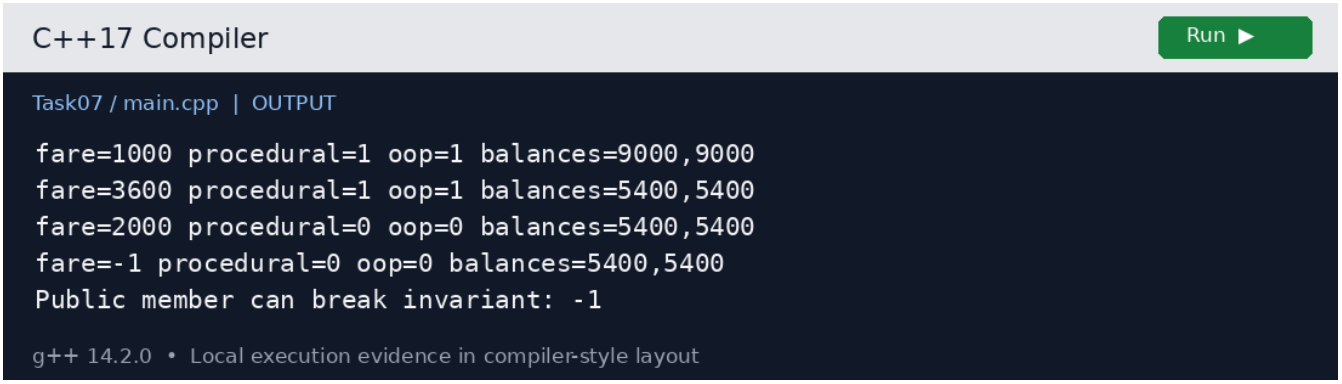
Task 7 From procedural to object oriented

Task07/main.cpp contains both working styles. The procedural version takes a caller-owned balance by reference and uses free canTravel and deduct functions. The OOP version instantiates SmartCard, keeps balance_ private, and exposes validated deduct(), canTravel() and recharge(). Their results agree for accepted, denied and negative-fare requests.

The guard checks fare >= 0, balance >= MIN_BALANCE and fare <= balance − MIN_BALANCE before subtracting. The ordering short-circuits safely and avoids calculating a potentially unsafe balance − fare. After a successful deduction the remaining amount is at least 5000; on failure the private value is unchanged. Constructor validation rejects a negative initial amount, and recharge checks LLONG_MAX − balance_ before addition.

Changing MIN_BALANCE in the common header changes both styles because both refer to the same constant. The OOP style is easier to enforce across many callers: they cannot bypass the deduction rule with direct assignment. Encapsulation alone does not make a multi-threaded transaction atomic; synchronization or transactional storage is still required in production.

Execution output



Task 7

Alternative concept and complexity

A public-member alternative is struct PublicCard { long long balance; }; with free functions or direct writes. It has the same O(1) operation time and 8-byte balance storage on this platform, but any caller can write balance = -1. The experiment demonstrates that exact invalid state. A nonvirtual class normally adds no per-instance overhead just because methods are private. Use composition and private storage to make invalid mutations harder to express.

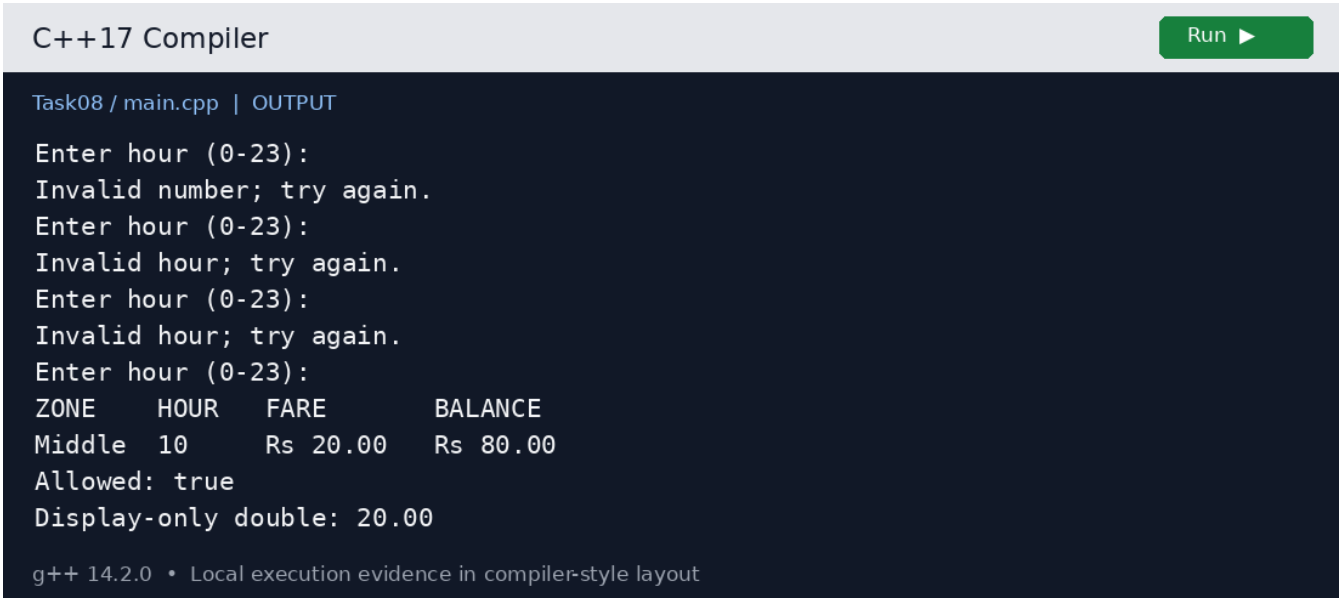
Task 8 Read and print data safely

readHour() repeatedly extracts an int with cin and checks the stream state. Letters set failbit; cin.clear() resets it and cin.ignore(max, newline) removes the bad line. If EOF ends input, return nullopt instead of looping forever. A successful extraction is followed by getline of the remainder, so 8x is rejected rather than silently accepted as 8. Values outside 0–23 are also rejected.

The supplied input is four lines: abc, 25, 8x and 10. The run recovers from each invalid line and accepts 10, producing a Middle off-peak fare of Rs 20.00 and a balance of Rs 80.00. Task09 separately verifies that letters trigger failure when reading an hour.

Money formatting uses integer quotient and remainder, with setw(2) and setfill('0') for exactly two paise digits. setfill is persistent and should be restored when later formatting unrelated fields. setw applies only to the next insertion. fixed and setprecision(2) are appropriate for a display-only double, but are not a substitute for integer accounting. The console report identifies zone, hour, fare, balance and acceptance.

Execution output



Task 8

Alternative concept and complexity

The line-based alternative reads a whole string with getline, then uses stoi(line, &used) inside try/catch. Reject invalid_argument and out_of_range, require the unconsumed suffix to contain only whitespace, and validate 0–23. Pseudocode: WHILE getline(line): TRY parse; validate consumed characters and range; RETURN hour; CATCH report bad input. Both approaches take O(L) time per L-character line including discarding its suffix. getline/stoi stores O(L) characters. Direct extraction can use O(1) application storage with ignore; this implementation also uses an O(L) tail string to reject trailing junk. Both can recover one line at a time.

Task 9 Implement and test the fare logic

The complete fare and card operations are in Common/metro.hpp, exercised by Task09/main.cpp. Expected values are literal independently calculated values. Each check compares expected with actual, prints PASS or FAIL, and the program exits nonzero if any check fails.

Case	Expected	Actual	Result
inner off	1000	1000	PASS
middle off	2000	2000	PASS

Case	Expected	Actual	Result
outer off	3000	3000	PASS
inner peak	1200	1200	PASS
middle peak	2400	2400	PASS
outer peak	3600	3600	PASS
08 boundary	3600	3600	PASS
10 boundary	3000	3000	PASS
17 boundary	3600	3600	PASS
20 boundary	3000	3000	PASS
student peak	3240	3240	PASS
senior peak	2880	2880	PASS
exact Rs50	1	1	PASS
exact balance	5000	5000	PASS
one paisa short	0	0	PASS
denial unchanged	5999	5999	PASS
negative recharge	0	0	PASS
negative fare	0	0	PASS
overflow recharge	0	0	PASS
unknown zone	1	1	PASS
hour 24	1	1	PASS

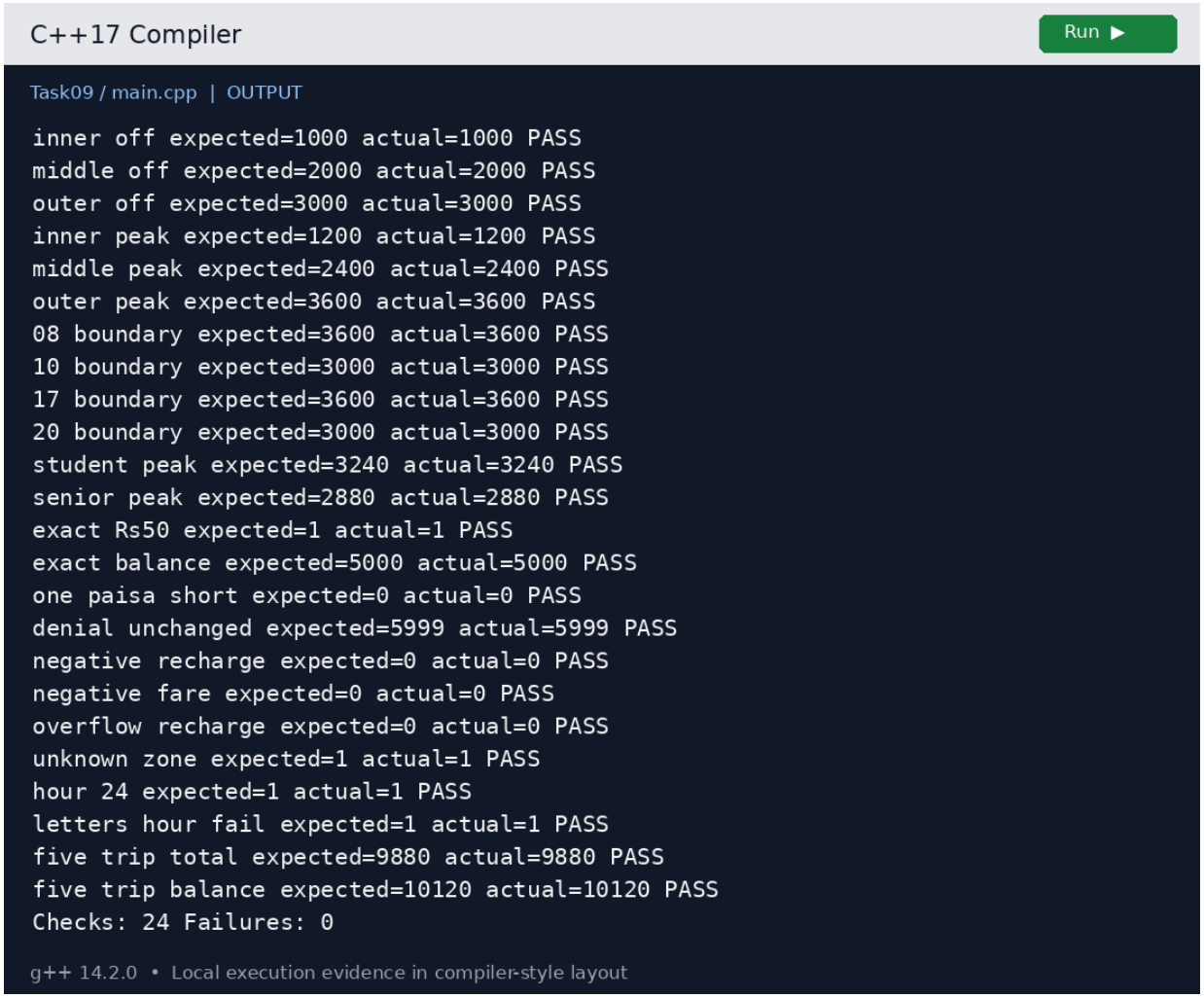
Case	Expected	Actual	Result
letters hour fail	1	1	PASS
five trip total	9880	9880	PASS
five trip balance	10120	10120	PASS

Fare and balance entries in the table are paise; Boolean results are 1 for true and 0 for false. The table includes every automated check. Unknown zones and invalid hours are rejected with an exception before quoting. Negative fares and recharges are rejected before mutation. The one-paisa-short test confirms that denial preserves the full original balance.

Five journeys calculated by hand

Journey	Calculation in paise	Fare
Inner adult off peak	1000	1000
Middle adult 08:00	2000 × 120 / 100	2400
Outer adult 10:00	3000	3000
Inner student 17:00	1200 × 90 / 100	1080
Outer senior 20:00	3000 × 80 / 100	2400
Total and final balance	1000 + 2400 + 3000 + 1080 + 2400	9880; 20000 − 9880 = 10120

Execution output



Task 9

Alternative concept and complexity

A table-driven approach stores {input, expected} records and iterates through them; that is used here for fares, with separate mutation checks for stateful operations. Running t cases costs O(t) time in both table-driven and handwritten approaches. The table costs O(t) storage, or O(1) auxiliary storage if cases stream in. Adding a new boundary is one record, whereas handwritten tests add another block that may duplicate comparison logic. The production quote itself remains O(1).

Task 10 Compare expectations with execution

For actual adult Outer peak fares of Rs 36.00, both double rupees and integer paise give exact totals at all three scales. To expose decimal representation error, a separate stress experiment repeatedly adds Rs

10.01, a hypothetical revised tariff rather than one of the base fares. The program measures the difference from the integer-paise total converted to rupees for comparison.

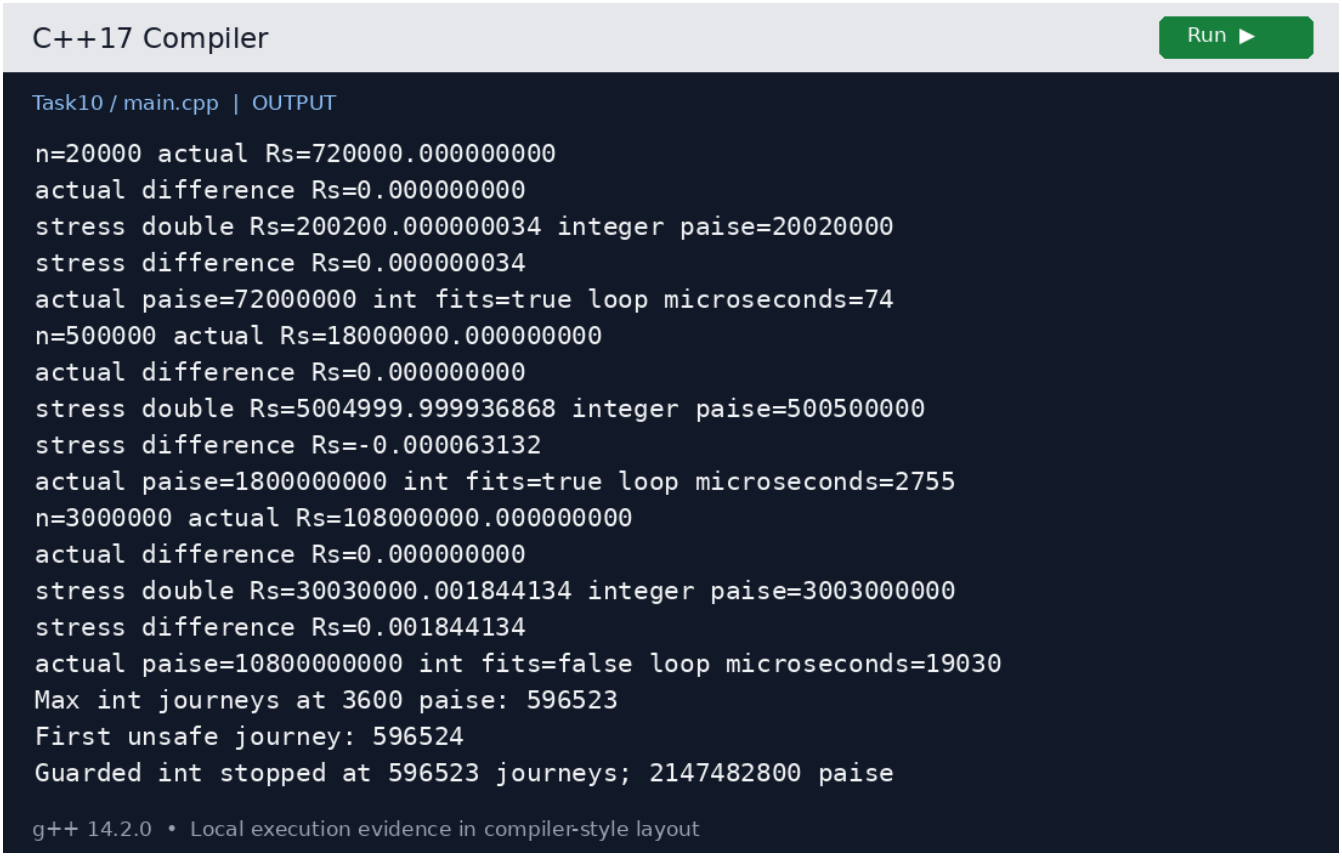
Journeys	Exact actual paise	Actual double Rs	Stress error Rs	Loop µs
20000	72,000,000	720000.000000000	0.000000034	74
500000	1,800,000,000	18000000.000000000	-0.000063132	2755
3000000	10,800,000,000	108000000.000000000	0.001844134	19030

The actual-fare difference is zero in every row. Stress exact totals are 20,020,000, 500,500,000 and 3,003,000,000 paise respectively. The largest measured stress error is about Rs 0.001844134 at 3 million additions. This is below one paisa in this run, but binary double does not ensure exact accounting for other values or scales. The sign and size of accumulated rounding depend on the values and sequence, so a simple claim that error always grows monotonically would be incorrect.

At 3600 paise per trip, int is safe through 596,523 journeys: 2,147,482,800 paise. Journey 596,524 would require 2,147,486,400 paise, above INT_MAX. The guarded experiment stops before overflow; no undefined overflowing int addition is executed. long long safely holds the 10,800,000,000-paise network total. A double has finite precision and eventually cannot distinguish adjacent paise even before its magnitude range is exhausted.

The only -Wall -Wextra warning in the runnable sources is the intentional signed/unsigned comparison in Task04/main.cpp. It signals conversion of negative s to unsigned. The brace narrowing example intentionally fails compilation. All other runnable sources compile without warnings under the recorded flags; absence of warnings alone does not prove correctness.

Execution output



Task 10

Alternative concept and complexity

For n additions, int, long long and double each need O(n) total time and O(1) accumulator space; one addition is O(1). Measured storage is 4, 8 and 8 bytes respectively. int fails the network range requirement in paise; long long remains exact within its range; double rupees are approximate for nonbinary fractions. A typical binary64 can represent integers exactly through 2^53; interpreting doubles as paise still requires checking that precision boundary. The displayed timings cover all three updates in one unoptimized loop, not isolated per-type speed comparisons.

Task 11 Recommend and defend the final design

Recommend long long paise for fare, balance and collection totals; int for validated hours and percentages; bool for decisions; scoped enums for zones and passenger categories; constexpr tables for a stable tariff; and private SmartCard storage with one mutation path. Use FareTable to quote prices and

Journey to carry input. This recommendation follows the precision example, the int capacity limit, the boundary tests and the 24 passing validation checks.

A fare revision changes one table entry or named percentage and triggers recompilation, regression tests and deployment. Shared rules mean that clients should not reproduce arithmetic in gate code. For the small original fare set, a constexpr table is simple and easy to audit. The supplied Task06 switch remains a valid teaching implementation; the final common design uses a validated table index.

The design is correct for a network-scale daily total under the tested bounds, but O(1) per journey does not establish throughput. Three million journeys still require O(n) work across a day. Multi-gate updates require atomic database transactions or synchronization; the demonstration has neither persistent storage nor concurrency control. Real deployment also needs authenticated recharge, card identifiers, audit trails, routing validation, clock and calendar services, rule versioning and recovery from interrupted deductions.

Configuration alternative implemented

Task11/main.cpp reads the zone count and fares from fares.txt, rejects missing, extra, out-of-range or nonpositive values, and prints the validated table. The allowed count is 1–100 and each fare is capped at 1,000,000 paise. A production loader should load into temporary storage and activate the new version only after validation, with an approved fallback on failure.

Execution output



Task 11

Alternative concept and complexity

Reading z fares from a configuration file takes O(z) startup time and O(z) table memory, followed by O(1) checked lookup. Compiled tables also occupy O(z) space but need no file parsing; tariff changes require rebuilding and redeploying. Configuration avoids recompiling code but introduces missing-file, malformed-value and authorization risks. A startup-only immutable validated configuration is a reasonable next step when tariffs change frequently. Task11 demonstrates loading; it is not yet wired into FareTable as a replacement policy.

Task 12 Re evaluate and adapt the design

A Impact analysis

Element	Required change
Constants and zone enum	Add Airport and 6000 paise; add WEEKEND_DISCOUNT = 10; expand three entries to four.
Expressions	Weekend chooses 90%, otherwise peak chooses 120%; concession follows. Never multiply both 90% and 120%.
Switch statements	A switch implementation needs Airport and validation updates. Checked array indexing needs only the new legal range and entry.
Journey and passenger	Add origin, destination, weekend flag and calendar month; preserve passenger category.
Classes	Add Pass with two distinct zones and validity month; replace quote policy. SmartCard reserve and money operations are retained.

B Evaluate the original design

The original switch can absorb one Airport case, but it cannot by itself represent pass validity or the priority between weekend and peak rules. Adding every combination as a case would duplicate policy logic. SmartCard can already deduct a zero or positive fare while maintaining its invariant, so changing

card storage is unnecessary. Retain the card and MODIFY the fare policy, rather than redesign the complete system.

Airport weekday peak is 6000 × 120 / 100 = 7200 paise. Airport weekend at 08:00 is 6000 × 90 / 100 = 5400 paise; it must not become 6480 by also adding the peak surcharge. An active Inner and Middle pass quotes zero for that covered route. A Middle to Outer trip falls outside that pass and costs 3600 paise at peak. A November pass used in December is not valid.

C Implement the adaptation

The revised namespace introduces a four-entry FARES table, Trip, Pass and FarePolicy. quote validates the route, hour, month and category before checking a pass. A covered pass returns zero; otherwise the policy selects one time-based multiplier and then applies the category concession. Pass members are private and its constructor rejects duplicate or unknown zones. Source is listed in Appendix B under Task12/main.cpp.

Pass coverage checks all radially ordered zones between origin and destination. Thus an Inner plus Outer pass does not cover a route that crosses Middle. Two zones need not be adjacent for separate journeys within either chosen zone, but a journey through an unselected intermediate zone is charged normally. The policy assumes valid calendar-month data; a real date service must derive weekend and month rather than trust operator-entered flags.

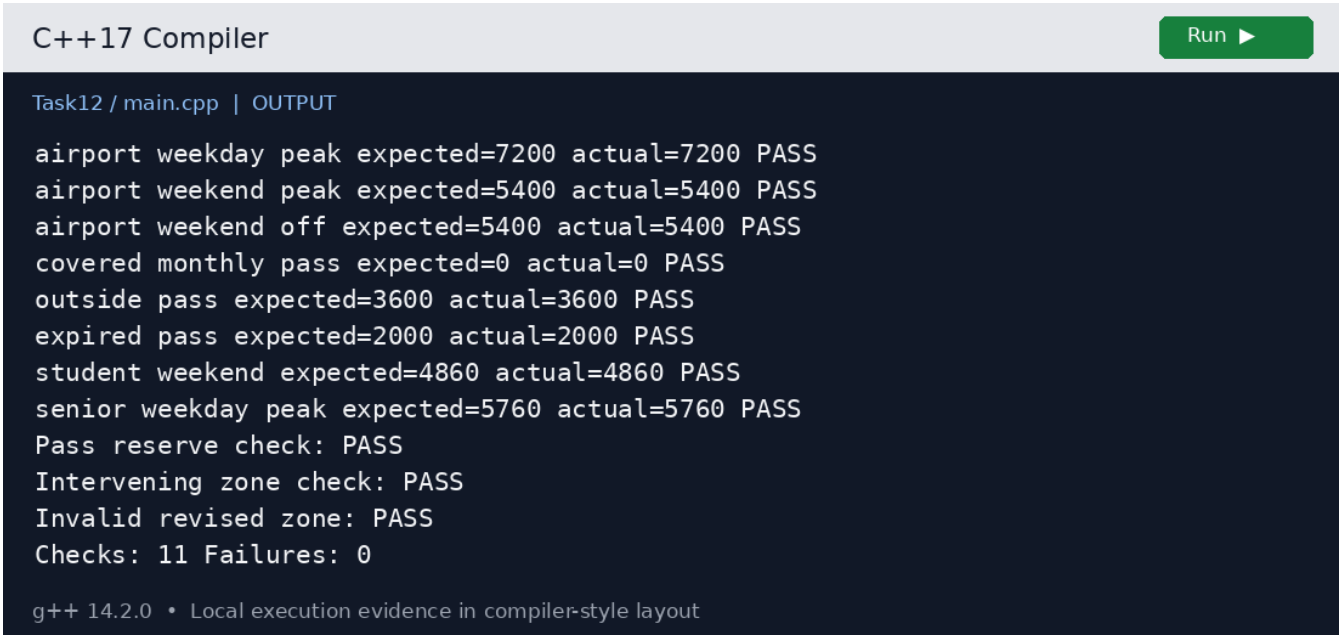
D Evidence based defence

Scenario	Expected paise	Actual paise	Result
airport weekday peak	7200	7200	PASS
airport weekend peak	5400	5400	PASS
airport weekend off	5400	5400	PASS
covered monthly pass	0	0	PASS
outside pass	3600	3600	PASS
expired pass	2000	2000	PASS

Scenario	Expected paise	Actual paise	Result
student weekend	4860	4860	PASS
senior weekday peak	5760	5760	PASS

Three additional checks pass: reserve enforcement on zero-fare travel, rejection of a route through an uncovered intermediate zone, and rejection of an unknown revised zone. Total: 11 checks, zero failures. The original 24 checks continue to pass because the extension is isolated from the original policy. This supports retaining card encapsulation while changing the tariff model.

Execution output



Task 12

Alternative concept and complexity

The original quote is O(1) in time and O(1) in memory for three fixed zones. The revised lookup and two-zone membership test are O(1) for the fixed four-zone system, with 32 bytes of base-fare entries on this platform instead of 24. The implemented coverage loop costs O(k) for k traversed zones; $k \leq 4$ here, so it is bounded. If generalized to arbitrary z, its worst-case time is O(z) and table space O(z). An arbitrary selected-zone set could use a bit mask or membership structure; that generality is unnecessary for this assignment. The policy adds complexity only where the requirements changed.

Appendix A Shared C plus plus implementation

Common/metro.hpp is included by the task programs. Copy the complete Assignment1_CO1_192572049 folder to preserve the relative include paths. All money computations below use paise. The standalone Task06 program intentionally retains its supplied constants and program structure.

```
#pragma once
#include <array>
#include <climits>
#include <iomanip>
#include <sstream>
#include <stdexcept>
#include <string>
namespace metro {
using Money = long long;
enum class Zone { Inner, Middle, Outer };
enum class Category { Adult, Student, Senior };
inline constexpr std::array<Money, 3> BASE{1000, 2000, 3000};
inline constexpr Money MIN_BALANCE{5000};
inline constexpr int PEAK_PERCENT{20};
inline constexpr int STUDENT_PERCENT{10}; // Assumed concession
inline constexpr int SENIOR_PERCENT{20};  // Assumed concession
inline std::size_t index(Zone z) {
    int n = static_cast<int>(z);
    if (n < 0 || n >= static_cast<int>(BASE.size()))
        throw std::invalid_argument("unknown zone");
    return static_cast<std::size_t>(n);
}
inline bool isPeak(int hour) {
    if (hour < 0 || hour > 23)
        throw std::invalid_argument("invalid hour");
    return (hour >= 8 && hour < 10) ||
```

```
        (hour >= 17 && hour < 20);
}
inline int concession(Category c) {
    switch (c) {
    case Category::Adult: return 0;
    case Category::Student: return STUDENT_PERCENT;
    case Category::Senior: return SENIOR_PERCENT;
    }
    throw std::invalid_argument("unknown category");
}
inline Money roundPercent(Money amount, int percent) {
    if (amount < 0 || percent < 0 || percent > 200 ||
        amount > (LLONG_MAX - 50) / 200)
        throw std::invalid_argument("percentage out of range");
    return (amount * percent + 50) / 100; // half up
}
struct Passenger { Category category{Category::Adult}; };
struct Journey { Zone zone; int hour; Passenger passenger{}; };
class FareTable {
public:
    Money quote(const Journey& j) const {
        Money base = BASE.at(index(j.zone));
        bool peak = isPeak(j.hour);
        int discount = concession(j.passenger.category);
        Money gross = roundPercent(base, peak ? 100 + PEAK_PERCENT : 100);
        return roundPercent(gross, 100 - discount);
    }
};
class SmartCard {
    Money balance_;
public:
    explicit SmartCard(Money balance) : balance_{balance} {
```

```
        if (balance < 0) throw std::invalid_argument("negative balance");
    }
    Money balance() const { return balance_; }
    bool canTravel(Money f) const {
        return f >= 0 && balance_ >= MIN_BALANCE &&
               f <= balance_ - MIN_BALANCE;
    }
    bool deduct(Money f) {
        if (!canTravel(f)) return false;
        balance_ -= f;
        return true;
    }
    bool recharge(Money value) {
        if (value <= 0 || value > LLONG_MAX - balance_) return false;
        balance_ += value;
        return true;
    }
};
inline std::string money(Money p) {
    if (p < 0) throw std::invalid_argument("negative money");
    std::ostringstream out;
    out << "Rs " << p / 100 << '.' << std::setw(2)
        << std::setfill('0') << p % 100;
    return out.str();
}
} // namespace metro
```

Appendix B Complete task source code

Each listing is the complete file. The task folder name is the location relative to the assignment folder. Programs are compiled with g++ -std=c++17 -Wall -Wextra. Task04/narrowing.cpp is the expected compile-time rejection example.

Task 1 main.cpp

File: Task01/main.cpp

```
#include "../Common/metro.hpp"
#include <iostream>
int main() {
    metro::Passenger student{metro::Category::Student};
    metro::Journey j{metro::Zone::Outer, 9, student};
    metro::FareTable table;
    metro::SmartCard card{10000};
    auto f = table.quote(j);
    bool ok = card.deduct(f);
    std::cout << "Student outer peak fare: " << metro::money(f)
              << "\nJourney allowed: " << std::boolalpha << ok
              << "\nBalance: " << metro::money(card.balance()) << '\n';
}
```

Task 2 main.cpp

File: Task02/main.cpp

```
#include "../Common/metro.hpp"
#include <iostream>
int main() {
    static_assert(metro::BASE.size() == 3);
    static_assert(metro::MIN_BALANCE == 5000);
    for (std::size_t i = 0; i < metro::BASE.size(); ++i)
        std::cout << "Zone " << i << ": "
                  << metro::money(metro::BASE[i]) << '\n';
```

```
    std::cout << "Peak surcharge: " << metro::PEAK_PERCENT << "%\n"
              << "Minimum remaining: "
              << metro::money(metro::MIN_BALANCE) << '\n';
}
```

Task 3 main.cpp

File: Task03/main.cpp

```
#include <climits>
#include <iomanip>
#include <iostream>
int main() {
    std::cout << "Type sizes in bytes\n"
      << "bool " << sizeof(bool) << " char " << sizeof(char)
      << " int " << sizeof(int) << " long long " << sizeof(long long)
      << "\nunsigned " << sizeof(unsigned) << " float " << sizeof(float)
      << " double " << sizeof(double) << '\n';
    auto count = 3000000LL;
    std::cout << "auto count " << sizeof(count) << '\n'
      << "CHAR_BIT " << CHAR_BIT << " CHAR " << CHAR_MIN << ".." << CHAR_MAX
      << "\nINT " << INT_MIN << ".." << INT_MAX
      << "\nUINT 0.." << UINT_MAX
      << "\nLLONG " << LLONG_MIN << ".." << LLONG_MAX << '\n';
    std::cout << std::setprecision(17)
      << "0.1 + 0.2 = " << 0.1 + 0.2 << '\n'
      << "10 + 20 paise = " << 10LL + 20LL << '\n'
      << "Network peak total paise: " << count * 3600LL << '\n'
      << "Fits int: " << std::boolalpha
      << (count * 3600LL <= INT_MAX) << '\n';
}
```

Task 4 main.cpp

File: Task04/main.cpp

```
#include <iostream>
int main()
{
int a = 7, b = 2;
double c = a / b;
double d = static_cast<double>(a) / b;
std::cout << c << ' ' << d << '\n';
char ch = 'A' + 2;
std::cout << ch << ' ' << ch + 1 << '\n';
bool flag = 5;
std::cout << flag << ' ' << std::boolalpha << flag << '\n';
unsigned int u = 1;
int s = -1;
std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

Task 4 explicit.cpp

File: Task04/explicit.cpp

```
#include <iostream>
int main() {
    int a{7}, b{2};
    double c{static_cast<double>(a / b)};
    double d{static_cast<double>(a) / static_cast<double>(b)};
    char ch{static_cast<char>(static_cast<int>('A') + 2)};
    bool flag{static_cast<bool>(5)};
    unsigned int u{1};
    int s{-1};
    std::cout << c << ' ' << d << '\n'
              << ch << ' ' << static_cast<int>(ch) + 1 << '\n'
```

```
        << flag << ' ' << std::boolalpha << flag << '\n'
        << (static_cast<unsigned int>(s) < u ? "s < u" : "s >= u")
        << '\n';
    // The cast above preserves the original unsigned comparison.
    // For intended mathematical comparison, widen both operands:
    std::cout << "Corrected comparison: "
      << (static_cast<long long>(s) < static_cast<long long>(u)) << '\n';
}
```

Task 4 narrowing.cpp

File: Task04/narrowing.cpp

```
// Deliberate compile-time failure; not a runnable program.
int main() { int x{3.7}; return x; }
```

Task 5 main.cpp

File: Task05/main.cpp

```
#include <iostream>
int main()
{
int x = 5, y = 3;
std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';
int z = x++ + ++y;
std::cout << z << ' ' << x << ' ' << y << '\n';
std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';
int k = 0;
for (int i = 0; i < 10; i += 3) k += i;
std::cout << k << '\n';
}
```

Task 5 peak_table.cpp

File: Task05/peak_table.cpp

```
#include "../Common/metro.hpp"
#include <array>
#include <iostream>
int main() {
    constexpr std::array<bool, 24> peak{
        false,false,false,false,false,false,false,false,
        true,true,false,false,false,false,false,false,
        false,true,true,true,false,false,false,false};
    int mismatches = 0;
    for (int h = 0; h < 24; ++h)
        if (peak.at(static_cast<std::size_t>(h)) != metro::isPeak(h))
            ++mismatches;
    std::cout << "Hours tested: 24\nMismatches: " << mismatches
              << "\nTable bytes: " << sizeof(peak) << '\n';
    return mismatches == 0 ? 0 : 1;
}
```

Task 6 main.cpp

File: Task06/main.cpp

```
#include <iomanip>
#include <iostream>
enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
// Rs 50 must remain on the card
long long fare(Zone z, bool peak)
{
// TODO 1 completed
```

```
long long base = 0;
switch (z) {
case Zone::Inner: base = BASE_PAISE[0]; break;
case Zone::Middle: base = BASE_PAISE[1]; break;
case Zone::Outer: base = BASE_PAISE[2]; break;
default: return -1;
}
return base + (peak ? base * PEAK_SURCHARGE_PERCENT / 100 : 0);
}
bool canTravel(long long balance, long long f)
{
// TODO 2 completed; avoid overflow in balance - f
return f >= 0 && balance >= MIN_BALANCE_PAISE &&
      f <= balance - MIN_BALANCE_PAISE;
}
int main()
{
long long balance = 10000;
// Rs 100.00 in paise
struct Trip { Zone zone; bool peak; };
Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
for (const Trip& t : trips) {
long long f = fare(t.zone, t.peak);
if (canTravel(balance, f)) {
balance -= f;
std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
<< std::setfill('0') << f % 100 << std::setfill(' ');
} else {
std::cout << "Denied (low balance)";
}
std::cout << " balance Rs " << balance / 100 << '\n';
}
```

```
}
```

Task 6 boundaries.cpp

File: Task06/boundaries.cpp

```
#include "../Common/metro.hpp"
#include <iostream>
int main() {
    metro::FareTable f;
    for (int h : {8, 10, 17, 20})
        std::cout << "Hour " << h << " peak=" << std::boolalpha
                  << metro::isPeak(h) << " fare="
                  << metro::money(f.quote({metro::Zone::Outer, h, {}}))
                  << '\n';
}
```

Task 7 main.cpp

File: Task07/main.cpp

```
#include "../Common/metro.hpp"
#include <iostream>
// Procedural version, with a caller-owned balance.
bool canTravel(long long balance, long long fare) {
    return fare >= 0 && balance >= metro::MIN_BALANCE &&
           fare <= balance - metro::MIN_BALANCE;
}
bool deduct(long long& balance, long long fare) {
    if (!canTravel(balance, fare)) return false;
    balance -= fare;
    return true;
}
int main() {
    long long balance = 10000;
    metro::SmartCard card{10000};
```

```
    for (long long fare : {1000LL, 3600LL, 2000LL, -1LL}) {
        bool p = deduct(balance, fare);
        bool o = card.deduct(fare);
        std::cout << "fare=" << fare << " procedural=" << p
                  << " oop=" << o << " balances=" << balance
                  << ',' << card.balance() << '\n';
        if (p != o || balance != card.balance()) return 1;
    }
    struct PublicCard { long long balance; };
    PublicCard unsafe{10000};
    unsafe.balance = -1;
    std::cout << "Public member can break invariant: "
              << unsafe.balance << '\n';
}
```

Task 8 main.cpp

File: Task08/main.cpp

```
#include "../Common/metro.hpp"
#include <algorithm>
#include <cctype>
#include <iostream>
#include <limits>
#include <optional>
std::optional<int> readHour() {
    for (;;) {
        std::cout << "Enter hour (0-23):\n";
        int hour{};
        if (!(std::cin >> hour)) {
            if (std::cin.eof()) return std::nullopt;
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << "Invalid number; try again.\n";
```

```
            continue;
        }
        std::string tail;
        std::getline(std::cin, tail);
        bool clean = std::all_of(tail.begin(), tail.end(),
            [](unsigned char c) { return std::isspace(c) != 0; });
        if (!clean || hour < 0 || hour > 23) {
            std::cout << "Invalid hour; try again.\n";
            continue;
        }
        return hour;
    }
}
int main() {
    auto h = readHour();
    if (!h) { std::cout << "Input ended.\n"; return 0; }
    metro::FareTable table;
    metro::SmartCard card{10000};
    auto fare = table.quote({metro::Zone::Middle, *h, {}});
    bool ok = card.deduct(fare);
    std::cout << "ZONE    HOUR   FARE      BALANCE\n"
              << "Middle  " << *h << "     " << metro::money(fare)
              << "    " << metro::money(card.balance()) << '\n'
              << "Allowed: " << std::boolalpha << ok << '\n';
    std::cout << std::fixed << std::setprecision(2)
              << "Display-only double: " << static_cast<double>(fare) / 100.0
              << '\n';
}
```

Task 9 main.cpp

File: Task09/main.cpp

```
#include "../Common/metro.hpp"
```

```
#include <iostream>
#include <sstream>
#include <vector>
using namespace metro;
int main() {
    FareTable table;
    struct Test { const char* name; Journey j; Money expected; };
    std::vector<Test> tests{
      {"inner off", {Zone::Inner, 11, {}}, 1000},
      {"middle off", {Zone::Middle, 11, {}}, 2000},
      {"outer off", {Zone::Outer, 11, {}}, 3000},
      {"inner peak", {Zone::Inner, 9, {}}, 1200},
      {"middle peak", {Zone::Middle, 9, {}}, 2400},
      {"outer peak", {Zone::Outer, 9, {}}, 3600},
      {"08 boundary", {Zone::Outer, 8, {}}, 3600},
      {"10 boundary", {Zone::Outer, 10, {}}, 3000},
      {"17 boundary", {Zone::Outer, 17, {}}, 3600},
      {"20 boundary", {Zone::Outer, 20, {}}, 3000},
      {"student peak", {Zone::Outer, 9, {Category::Student}}, 3240},
      {"senior peak", {Zone::Outer, 9, {Category::Senior}}, 2880}};
    int failures = 0, checks = 0;
    auto check = [&](const std::string& name, Money expected, Money actual) {
        ++checks;
        if (expected != actual) ++failures;
        std::cout << name << " expected=" << expected << " actual=" << actual
                  << (expected == actual ? " PASS\n" : " FAIL\n");
    };
    for (const auto& t : tests)
        check(t.name, t.expected, table.quote(t.j));
    SmartCard exact{6000}, low{5999};
    check("exact Rs50", 1, exact.deduct(1000));
    check("exact balance", 5000, exact.balance());
```

```
    check("one paisa short", 0, low.deduct(1000));
    check("denial unchanged", 5999, low.balance());
    check("negative recharge", 0, low.recharge(-100));
    check("negative fare", 0, low.deduct(-1));
    check("overflow recharge", 0, low.recharge(LLONG_MAX));
    bool rejected = false;
    try { table.quote({static_cast<Zone>(99), 10, {}}); }
    catch (const std::invalid_argument&) { rejected = true; }
    check("unknown zone", 1, rejected);
    rejected = false;
    try { table.quote({Zone::Inner, 24, {}}); }
    catch (const std::invalid_argument&) { rejected = true; }
    check("hour 24", 1, rejected);
    std::istringstream bad{"letters"};
    int hour{};
    bad >> hour;
    check("letters hour fail", 1, bad.fail());
    SmartCard batch{20000};
    const Journey trips[]{{Zone::Inner, 11, {}}, {Zone::Middle, 8, {}},
      {Zone::Outer, 10, {}}, {Zone::Inner, 17, {Category::Student}},
      {Zone::Outer, 20, {Category::Senior}}};
    Money total = 0;
    for (const auto& j : trips) {
        Money f = table.quote(j);
        if (!batch.deduct(f)) return 1;
        total += f;
    }
    check("five trip total", 9880, total);
    check("five trip balance", 10120, batch.balance());
    std::cout << "Checks: " << checks << " Failures: " << failures << '\n';
    return failures == 0 ? 0 : 1;
}
```

Task 10 main.cpp

File: Task10/main.cpp

```
#include <chrono>
#include <climits>
#include <iomanip>
#include <iostream>
int main() {
    std::cout << std::fixed << std::setprecision(9);
    for (long long n : {20000LL, 500000LL, 3000000LL}) {
        double actualFareTotal = 0.0, stressTotal = 0.0;
        long long paiseTotal = 0;
        auto start = std::chrono::steady_clock::now();
        for (long long i = 0; i < n; ++i) {
            actualFareTotal += 36.0;
            stressTotal += 10.01; // Precision stress, not a base fare
            paiseTotal += 1001;
        }
        auto us = std::chrono::duration_cast<std::chrono::microseconds>(
            std::chrono::steady_clock::now() - start).count();
        long long realPaise = n * 3600LL;
        std::cout << "n=" << n << " actual Rs=" << actualFareTotal << '\n'
          << "actual difference Rs="
          << actualFareTotal - static_cast<double>(realPaise) / 100.0 << '\n'
          << "stress double Rs=" << stressTotal
          << " integer paise=" << paiseTotal << '\n'
          << "stress difference Rs="
          << stressTotal - static_cast<double>(paiseTotal) / 100.0 << '\n'
          << "actual paise=" << realPaise << " int fits="
          << std::boolalpha << (realPaise <= INT_MAX)
          << " loop microseconds=" << us << '\n';
    }
```

```
    std::cout << "Max int journeys at 3600 paise: " << INT_MAX / 3600 << '\n'
      << "First unsafe journey: " << INT_MAX / 3600 + 1 << '\n';
    int sum = 0;
    long long accepted = 0;
    while (sum <= INT_MAX - 3600) { sum += 3600; ++accepted; }
    std::cout << "Guarded int stopped at " << accepted << " journeys; "
              << sum << " paise\n";
}
```

Task 11 main.cpp

File: Task11/main.cpp

```
#include "../Common/metro.hpp"
#include <fstream>
#include <iostream>
#include <vector>
int main() {
    std::ifstream in{"fares.txt"};
    int zones{};
    if (!(in >> zones) || zones < 1 || zones > 100) return 1;
    std::vector<metro::Money> fares(static_cast<std::size_t>(zones));
    for (auto& f : fares)
        if (!(in >> f) || f <= 0 || f > 1000000) return 1;
    std::string extra;
    if (in >> extra) return 1;
    if (!in.eof()) return 1;
    std::cout << "Validated configuration zones: " << fares.size() << '\n';
    for (std::size_t i = 0; i < fares.size(); ++i)
        std::cout << "Zone " << i << ": " << metro::money(fares[i]) << '\n';
    std::cout << "Startup O(z), quote O(1), table space O(z)\n";
}
```

Task 12 main.cpp

File: Task12/main.cpp

```
#include "../Common/metro.hpp"
#include <algorithm>
#include <array>
#include <iostream>
#include <optional>
namespace revised {
using metro::Money;
enum class Zone { Inner, Middle, Outer, Airport };
constexpr std::array<Money, 4> FARES{1000, 2000, 3000, 6000};
constexpr int WEEKEND_DISCOUNT{10};
int index(Zone z) {
    int i = static_cast<int>(z);
    if (i < 0 || i >= static_cast<int>(FARES.size()))
        throw std::invalid_argument("unknown revised zone");
    return i;
}
struct Trip {
    Zone from, to;
    int hour;
    bool weekend;
    int month; // YYYYMM, 1..12 months
    metro::Category category{metro::Category::Adult};
};
void validMonth(int m) {
    if (m < 200001 || m > 999912 || m % 100 < 1 || m % 100 > 12)
        throw std::invalid_argument("invalid month");
}
class Pass {
    Zone first_, second_;
```

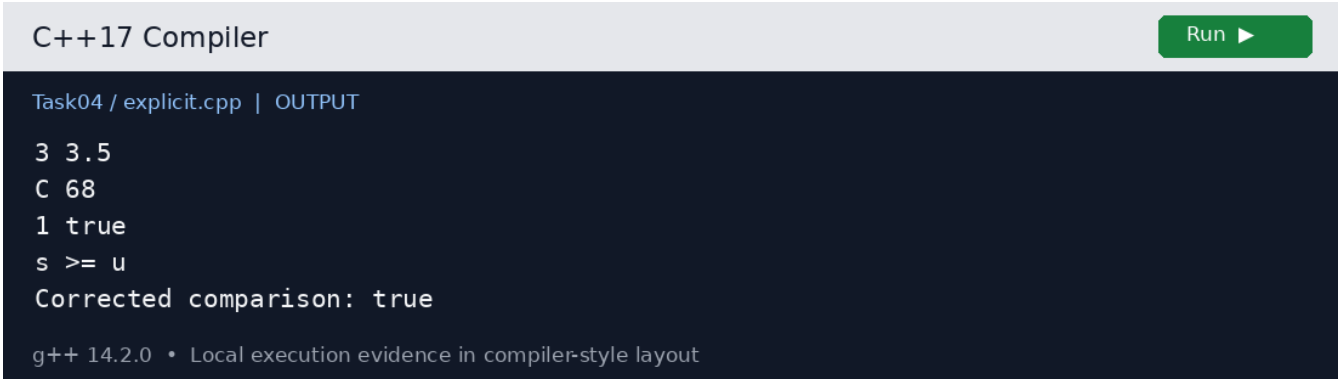
```
    int month_;
public:
    Pass(Zone a, Zone b, int m) : first_{a}, second_{b}, month_{m} {
        index(a); index(b); validMonth(m);
        if (a == b) throw std::invalid_argument("choose distinct zones");
    }
    bool covers(const Trip& t) const {
        if (month_ != t.month) return false;
        int lo = std::min(index(t.from), index(t.to));
        int hi = std::max(index(t.from), index(t.to));
        // Every traversed zone must be one of the two pass zones.
        for (int z = lo; z <= hi; ++z)
            if (z != index(first_) && z != index(second_)) return false;
        return true;
    }
};
class FarePolicy {
public:
    Money quote(const Trip& t, const std::optional<Pass>& pass) const {
        int from = index(t.from), to = index(t.to);
        bool peak = metro::isPeak(t.hour);
        validMonth(t.month);
        int concession = metro::concession(t.category);
        if (pass && pass->covers(t)) return 0;
        Money base = FARES.at(static_cast<std::size_t>(std::max(from, to)));
        int multiplier = t.weekend ? 100 - WEEKEND_DISCOUNT :
                         (peak ? 100 + metro::PEAK_PERCENT : 100);
        Money gross = metro::roundPercent(base, multiplier);
        return metro::roundPercent(gross, 100 - concession);
    }
};
} // namespace revised
```

```
int main() {
    using namespace revised;
    FarePolicy policy;
    std::optional<Pass> pass{Pass{Zone::Inner, Zone::Middle, 202611}};
    struct Test { const char* name; Trip trip; bool usePass; Money expected; };
    const Test tests[]{
      {"airport weekday peak", {Zone::Inner,Zone::Airport,8,false,202611},false,7200},
      {"airport weekend peak", {Zone::Inner,Zone::Airport,8,true,202611},false,5400},
      {"airport weekend off", {Zone::Inner,Zone::Airport,10,true,202611},false,5400},
      {"covered monthly pass", {Zone::Inner,Zone::Middle,8,false,202611},true,0},
      {"outside pass", {Zone::Middle,Zone::Outer,8,false,202611},true,3600},
      {"expired pass", {Zone::Inner,Zone::Middle,10,false,202612},true,2000},
      {"student weekend", {Zone::Inner,Zone::Airport,8,true,202611,
                           metro::Category::Student},false,4860},
      {"senior weekday peak", {Zone::Inner,Zone::Airport,8,false,202611,
                               metro::Category::Senior},false,5760}};
    int failures = 0;
    for (const auto& t : tests) {
        Money actual = policy.quote(t.trip, t.usePass ? pass : std::nullopt);
        bool ok = actual == t.expected;
        failures += !ok;
        std::cout << t.name << " expected=" << t.expected << " actual="
                  << actual << (ok ? " PASS\n" : " FAIL\n");
    }
    metro::SmartCard low{4999}, exact{5000};
    bool reserveOK = !low.deduct(0) && exact.deduct(0);
    failures += !reserveOK;
    std::cout << "Pass reserve check: " << (reserveOK ? "PASS" : "FAIL") << '\n';
    Pass separated{Zone::Inner, Zone::Outer, 202611};
    bool gapOK = !separated.covers({Zone::Inner,Zone::Outer,8,false,202611});
    failures += !gapOK;
    std::cout << "Intervening zone check: " << (gapOK ? "PASS" : "FAIL") << '\n';
```

```
    bool invalidOK = false;
    try { policy.quote({static_cast<Zone>(9),Zone::Inner,8,false,202611},pass); }
    catch (const std::invalid_argument&) { invalidOK = true; }
    failures += !invalidOK;
    std::cout << "Invalid revised zone: " << (invalidOK ? "PASS" : "FAIL") << '\n';
    std::cout << "Checks: 11 Failures: " << failures << '\n';
    return failures == 0 ? 0 : 1;
}
```

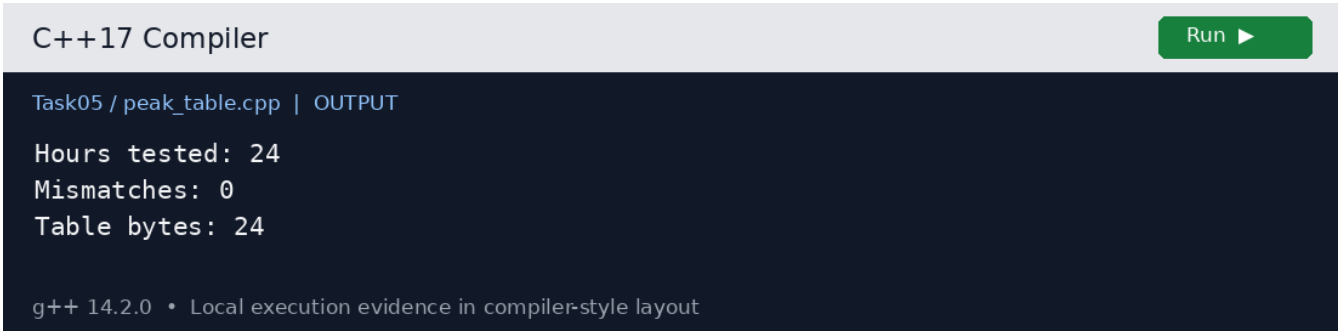
Appendix C Supplementary execution evidence

Execution output



Explicit conversions retain original output and then correct the comparison

Execution output



All 24 lookup-table entries agree with the expression